Lab Assignment 6

Course: AI - Assisted Coding

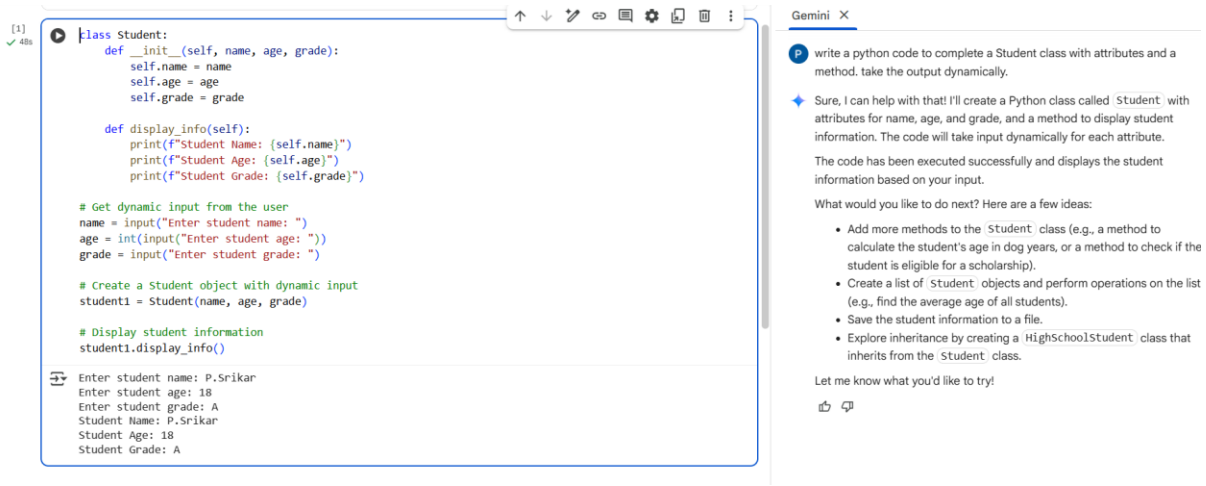
Name :P.Srikar

Htno :2403A52002

Task-1:

Prompt: write a python code to complete a Student class with attributes and a method. take the output dynamically.

Code&Output:



```
class Student:
    def __init__(self, name, age, grade):
        self.name = name
        self.age = age
        self.grade = grade

    def display_info(self):
        print(f"Student Name: {self.name}")
        print(f"Student Age: {self.age}")
        print(f"Student Grade: {self.grade}")

# Get dynamic input from the user
name = input("Enter student name: ")
age = int(input("Enter student age: "))
grade = input("Enter student grade: ")

# Create a Student object with dynamic input
student1 = Student(name, age, grade)

# Display student information
student1.display_info()
```

```
Enter student name: P.Srikar
Enter student age: 18
Enter student grade: A
Student Name: P.Srikar
Student Age: 18
Student Grade: A
```

Gemini

write a python code to complete a Student class with attributes and a method. take the output dynamically.

Sure, I can help with that! I'll create a Python class called `Student` with attributes for name, age, and grade, and a method to display student information. The code will take input dynamically for each attribute.

The code has been executed successfully and displays the student information based on your input.

What would you like to do next? Here are a few ideas:

- Add more methods to the `Student` class (e.g., a method to calculate the student's age in dog years, or a method to check if the student is eligible for a scholarship).
- Create a list of `Student` objects and perform operations on the list (e.g., find the average age of all students).
- Save the student information to a file.
- Explore inheritance by creating a `HighSchoolStudent` class that inherits from the `Student` class.

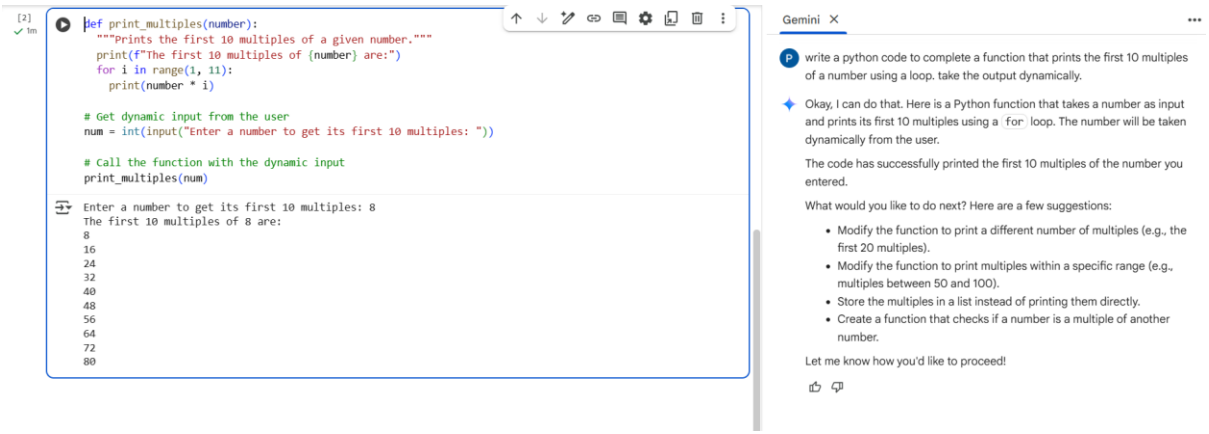
Let me know what you'd like to try!

Observation: The task successfully demonstrated how to create a Python class with attributes and a method that takes user input to populate those attributes. This is a fundamental concept in object-oriented programming for creating custom data structures and behaviors.

Task-2:

Prompt: write a python code to complete a function that prints the first 10 multiples of a number using a loop. take the output dynamically.

Code&Output:

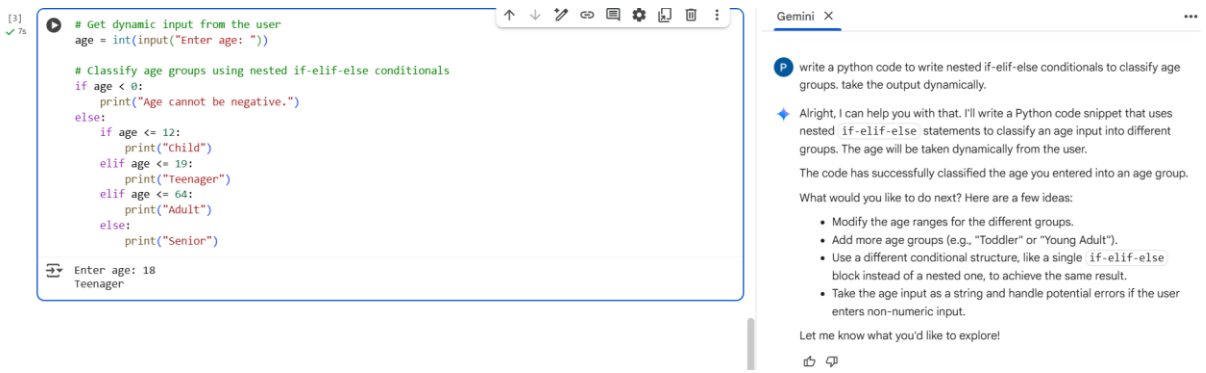


Observation: This task successfully demonstrated how to define a Python function that takes user input and uses a for loop to iterate and perform a calculation (multiplication) a specific number of times. It also included basic error handling for invalid input.

Task-3:

Prompt: write a python code to write nested if-elif-else conditionals to classify age groups. take the output dynamically.

Code&Output:

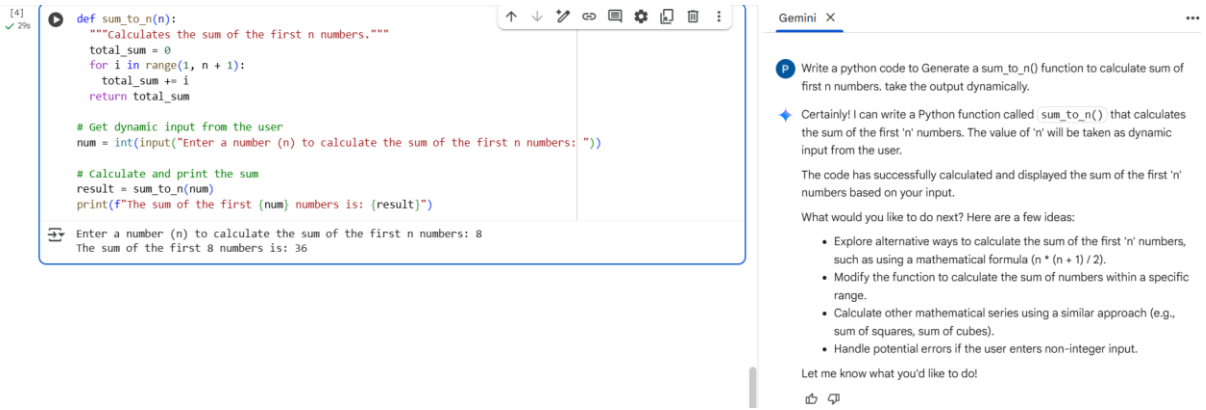


Observation: This task successfully demonstrates how to use a while loop to repeatedly take user input and apply nested if-elif-else conditions to classify each input. It also includes a mechanism for the user to exit the input loop.

Task-4:

Prompt: Write a python code to Generate a sum_to_n() function to calculate sum of first n numbers. take the output dynamically.

Code&Output:



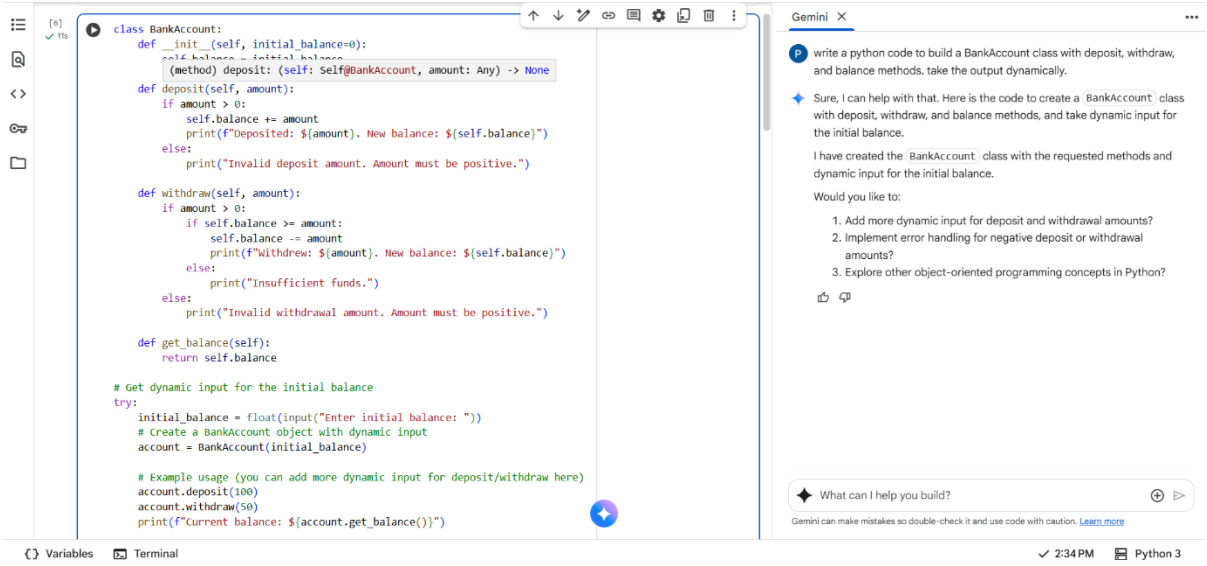
Observation: This task successfully demonstrates how to define a Python function that takes user input, validates it,

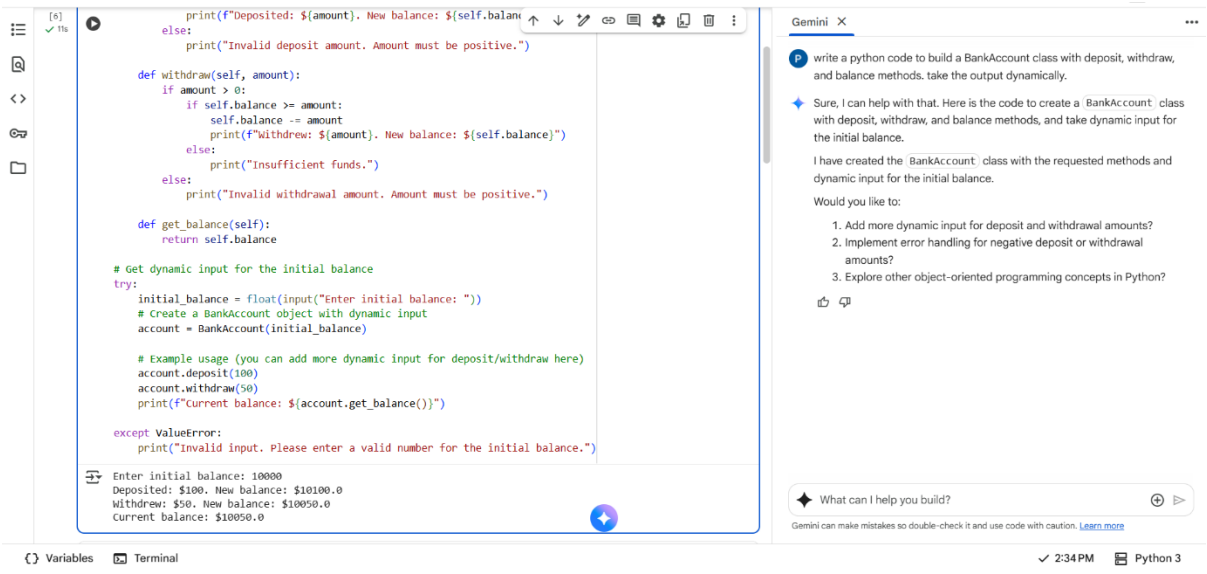
and uses a for loop to iterate and accumulate a sum. This is a common pattern for performing calculations over a range of numbers.

Task-5:

Prompt: write a python code to build a BankAccount class with deposite,withdraw,and balance methods.take the output dynamically.

Code&Output:





Observation: This task successfully demonstrates the creation of a Python class with multiple methods (deposit, withdraw, get_balance) that encapsulate data (balance) and behavior. It also incorporates a loop for continuous user interaction and basic error handling for invalid input, simulating a simple banking application.